Comprehensive OOP Interview Guide

Curated concepts and advanced paradigms for software engineering interviews.

Note to Candidate: This guide synthesizes core principles, architectural concepts, and advanced Object-Oriented Programming (OOP) topics to prepare you for technical interviews.

Core Principles of Object-Oriented Programming

1. What is Object-Oriented Programming (OOP), and how does it differ from Procedural Programming?

Object-Oriented Programming is a paradigm that organizes software around objects instead of functions. An object combines data (attributes) and methods (functions) into a single unit. It emphasizes inheritance, encapsulation, polymorphism, and abstraction.

- **Procedural Programming:** Focuses on functions. Data and functions are separate. It is generally less secure and better suited for smaller programs (e.g., C).
- **OOP:** Focuses on objects. Data and methods are bundled together. It is more secure due to encapsulation and is highly scalable for large applications (e.g., Java, C++, Python).

2. Explain the four pillars of OOP.

- **Abstraction:** Hiding complex implementation details and showing only the necessary features to the user. Achieved through abstract classes and interfaces.
- **Encapsulation:** Bundling data and methods that operate on the data within a single unit (class). Provides data security by preventing unauthorized access.
- **Inheritance:** A mechanism where a new class (subclass) derives from an existing class (superclass), inheriting properties and behaviors.
- **Polymorphism:** The ability of an object to take on many forms. Enables a single interface to represent multiple types of objects (via overloading and overriding).

3. What is the difference between Encapsulation and Data Abstraction?

While often used together, they serve different purposes:

- **Encapsulation (Data Hiding):** The mechanism that binds code and the data it manipulates into a single unit. Variables are kept private and accessed via public methods.
- **Data Abstraction (Detail Hiding):** Showing only essential details to the user while trivial

Classes and Objects

4. What is the difference between a class and an object?

A class is a logical blueprint or template for creating objects, defining a set of attributes and methods common to all objects of that type. An object is a physical entity and a specific instance of a class. It holds its own actual state (values for the attributes) and behavior. A class is like an architectural blueprint, while an object is the physical building.

5. How much memory does a class occupy?

Classes themselves do not occupy memory. They merely serve as a logical template. Memory is only allocated when an object (an instance of the class) is created and initializes the class members.

6. Is it always necessary to create objects from a class?

No. If a class contains non-static methods, an object must be constructed to invoke them. However, if the class exclusively contains static methods, you do not need to generate objects; you can call those methods directly using the class name.

7. What is the difference between a structure and a class in C++?

Both are user-defined data types. The major difference is default visibility: in a structure, members are **public** by default, whereas in a class, members are **private** by default.

Inheritance & Polymorphism

8. What is Inheritance, what are its types, and does it have any limitations?

Inheritance allows a subclass to acquire properties from a superclass, promoting code reuse.

- **Types:** Single, Multiple (supported in C++), Multilevel, Hierarchical, and Hybrid.
- **Limitations:** It creates tightly coupled classes. Changes in a base class often require simultaneous changes in derived classes, and deep hierarchies can impact processing time.

9. What is Polymorphism and what are its types?

Polymorphism allows code to behave differently based on the context.

- **Compile-Time (Static):** Method call is resolved during compilation (e.g., method/operator overloading).
- **Runtime (Dynamic):** The actual implementation is determined during execution based on the object type (e.g., overriding via virtual functions).

Inheritance & Polymorphism (Continued)

10. What is the difference between method overloading and method overriding?

- **Overloading:** Occurs when multiple methods in the same class share the same name but have different parameter lists. Resolved at compile time.
- **Overriding:** Occurs when a subclass provides a specific, new implementation for a method defined in its superclass. Resolved at runtime based on the actual object type.

11. What is an abstract class, and how does it differ from an interface?

An **abstract class** cannot be instantiated and is strictly a base class. It can contain both implemented and abstract methods, instance variables, and constructors. An **interface** is a collection of abstract methods acting as a strict behavioral contract. A class can inherit from only one abstract class but can implement multiple interfaces.

12. What are virtual functions and pure virtual functions?

- **Virtual Function:** A method in a base class that can be overridden in a derived class. It enables dynamic dispatch ensuring the correct method is called at runtime.
- **Pure Virtual Function:** An abstract function declared in a base class with no implementation. Subclasses must implement this to become instantiable.

13. Explain dynamic method dispatch.

It is the mechanism that enables a method call on an object to be resolved at runtime based on the actual type of the object, rather than the reference type pointing to it.

14. What is method hiding?

Method hiding occurs when a subclass defines a static method with the exact same name and signature as a static method in the superclass. The method invoked depends entirely on the reference type used at compile time, not the object type at runtime.

Constructors & Destructors

15. What is a constructor, and what are the different types?

A constructor initializes the state of a newly instantiated object.

- **Default:** Takes no arguments.
- **Parameterized:** Takes arguments to initialize with specific values.

Constructors & Destructors (Continued)

16. What is the difference between a public and private constructor?

A public constructor allows object instantiation from anywhere. A private constructor restricts object creation exclusively to within its own class. Private constructors are utilized in the Singleton design pattern to ensure only one instance of a class exists.

17. What is constructor chaining?

The process of calling one constructor from another within the same class (e.g., using `this()`) or between base and derived classes (e.g., using `super()`). It centralizes initialization logic.

18. What is a destructor, and can it be overloaded?

A destructor is called when an object is destroyed or goes out of scope to release resources (memory, files). A destructor takes no arguments, has no return type, and **cannot** be overloaded. A class can have only one destructor.

State, Modifiers & Keywords

19. What is the difference between static and non-static members?

- **Static:** Belong to the class itself. Shared across all objects and accessed via the class name.
- **Non-Static:** Belong to specific instances. Require an object to be invoked and can access both static and non-static data.

20. What is the role of the "this" keyword?

The `this` keyword is a reference to the current object instance. It is used to access instance variables and methods, disambiguating instance variables from local parameters with the same name.

21. What is the "super" keyword?

It is used within a subclass to call the superclass's constructor or to access the superclass's overridden methods and variables.

22. What are access modifiers?

Access modifiers define the visibility of class members:

- **Public:** Accessible from anywhere.
- **Private:** Accessible only within the same class.

State, Modifiers & Keywords (Continued)

23. Explain the "final" keyword.

When applied to a class, it prevents subclassing. When applied to a method, it prevents overriding. When applied to a variable, it turns it into a constant that cannot be reassigned.

24. What is the "instanceof" operator?

It tests whether an object is an instance of a particular class or implements a specific interface at runtime, returning a boolean value.

Advanced Architectural Concepts

25. What is the difference between a shallow copy and a deep copy?

- **Shallow Copy:** Copies the memory references of the original object's nested data. Altering nested data in one affects the other.
- **Deep Copy:** Creates a completely independent object and recursively copies all fields into new memory locations.

26. What is Coupling in OOP?

Coupling measures the interdependence between two classes. High coupling means classes are heavily dependent (changes cascade). Low (loose) coupling is ideal, keeping classes independent and easier to maintain.

27. Differentiate between Association, Aggregation, and Composition.

- **Association:** A general relationship where two objects are connected but exist independently.
- **Aggregation:** A weak "has-a" relationship where the child object can exist independently of the parent (e.g., Department and Professors).
- **Composition:** A strong "has-a" relationship where the child object's lifetime strictly depends on the parent object (e.g., House and Rooms).

Advanced Architectural Concepts (Continued)

28. What is the difference between Composition and Inheritance?

Composition represents a "has-a" relationship, while Inheritance represents an "is-a" relationship. Composition is generally favored over inheritance for code reuse as it avoids the tight coupling inherent in deep class hierarchies.

29. What is the Diamond Problem, and how is it resolved?

An ambiguity arising in multiple inheritance when a subclass inherits from two different parent classes that share a common base class. The compiler cannot determine which path to follow. In C++, it is resolved using `virtual` inheritance. Java avoids this by disallowing multiple inheritance of classes entirely.

30. What are friend functions and friend classes?

In C++, a `friend` function or class is granted special access to the `private` and `protected` members of another class in which it is declared, circumventing standard encapsulation rules for specific trusted entities.

31. What is the Liskov Substitution Principle (LSP)?

A core SOLID design principle stating that objects of a superclass should be completely replaceable with objects of any of its subclasses without altering the correctness or intended behavior of the program.